

CS 322

Camera Geometry

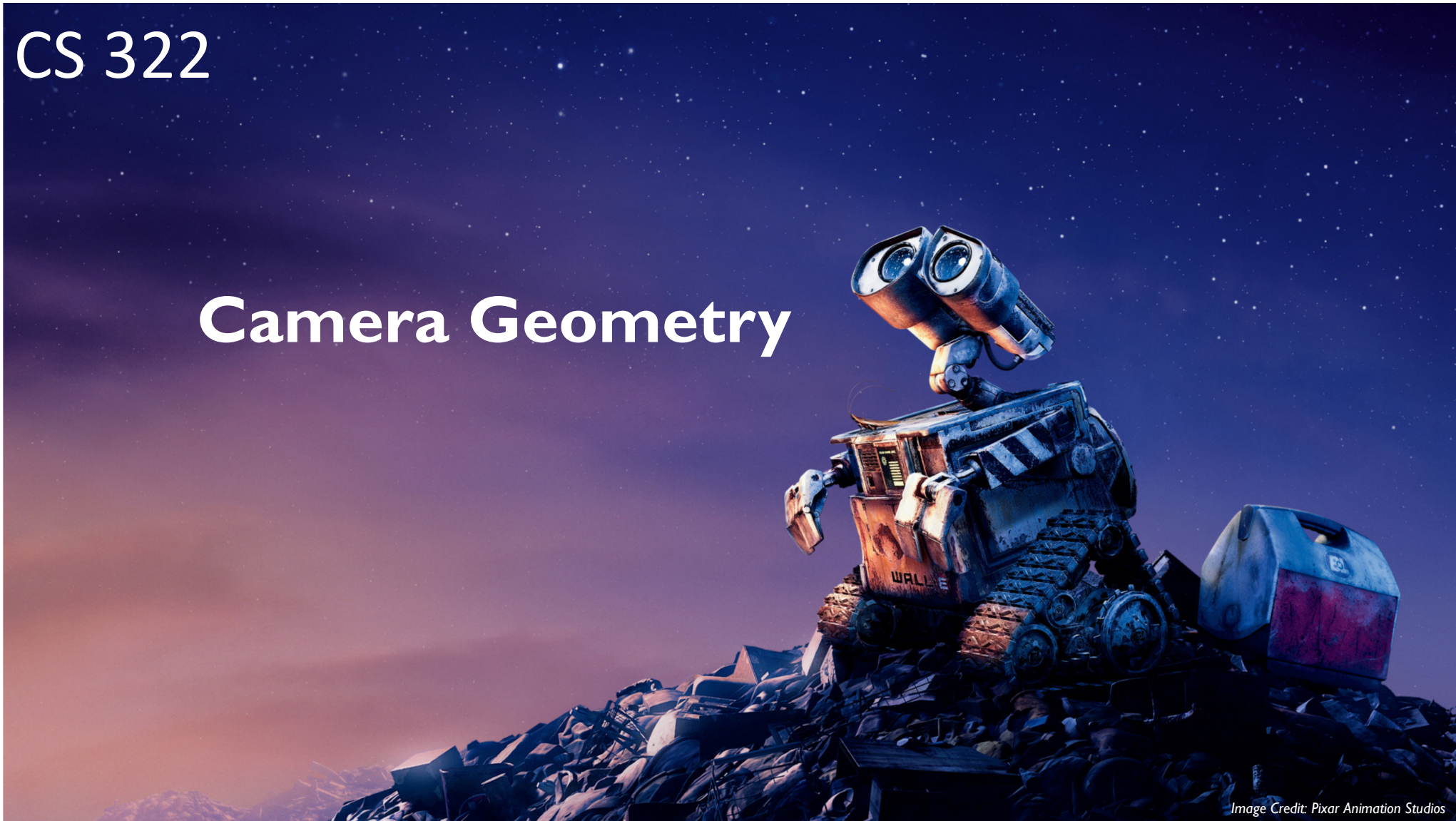


Image Credit: Pixar Animation Studios






Translate
Turn off instant translation


French English **Italian** Detect language ▼

↔ English French Italian ▼

Translate

camera

6/5000

room

Synonyms of camera

noun

vano, camera da letto

4 more synonyms

See also

camera da letto, camera doppia, camera singola, servizio in camera, camera d'aria, camera oscura, camera libera, camera mortuaria, camera dei bambini, camera con colazione

Translations of camera

noun

room

camera, stanza, sala, ambiente, spazio, locale

chamber

camera, cavità, aula

house

casa, abitazione, edificio, dimora, camera, albergo

apartment

appartamento, alloggio, camera, stanza

lodging

alloggio, alloggiamento, appartamento, camera

Google Translate for Business:
[Translator Toolkit](#)
[Website Translator](#)
[Global Market Finder](#)

Camera Obscura: dark room

Known during classical period in China and Greece (e.g., Mo-Ti, China, 470BC to 390BC)

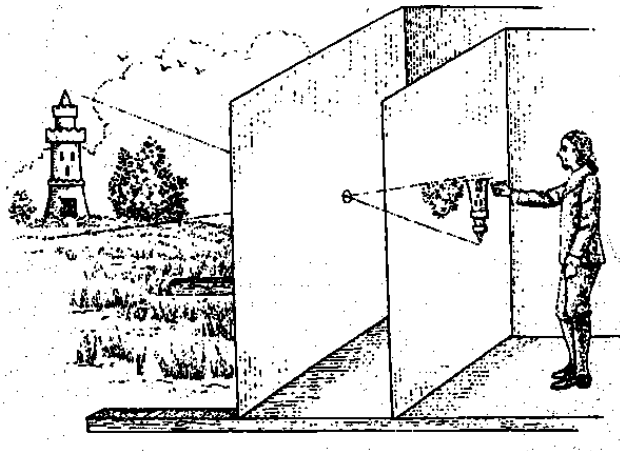


Illustration of Camera Obscura



Freestanding camera obscura at UNC Chapel Hill

Photo by Seth Ilys

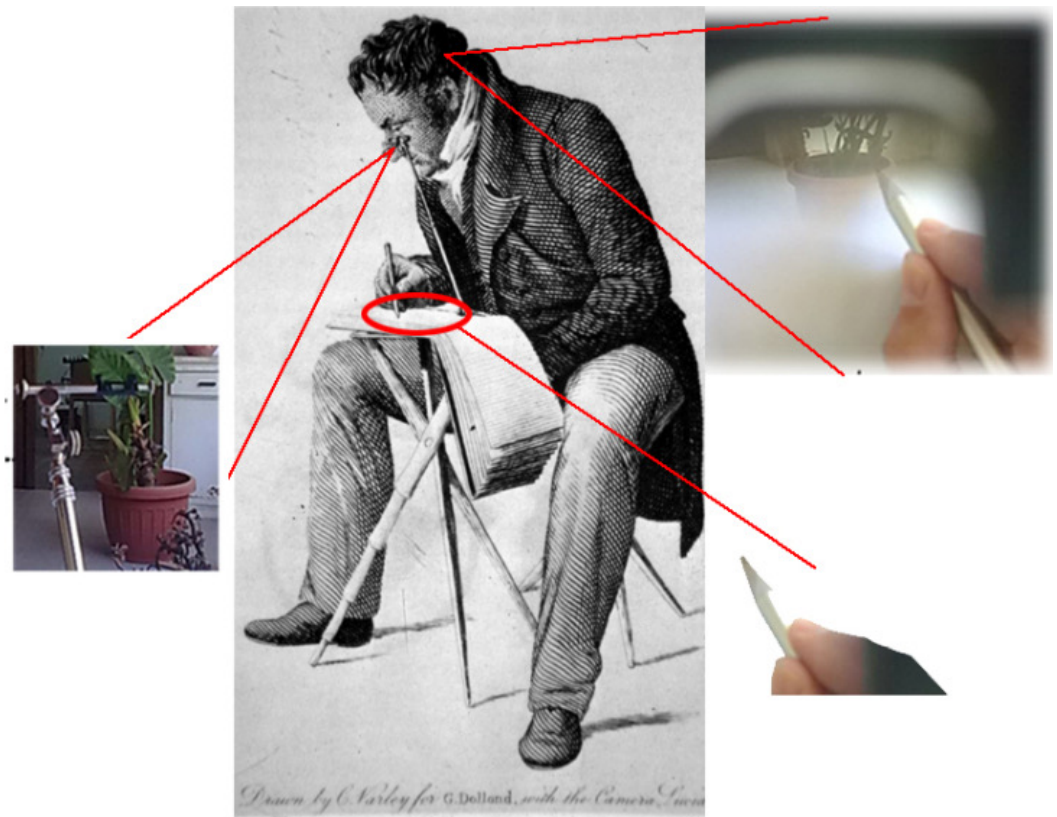
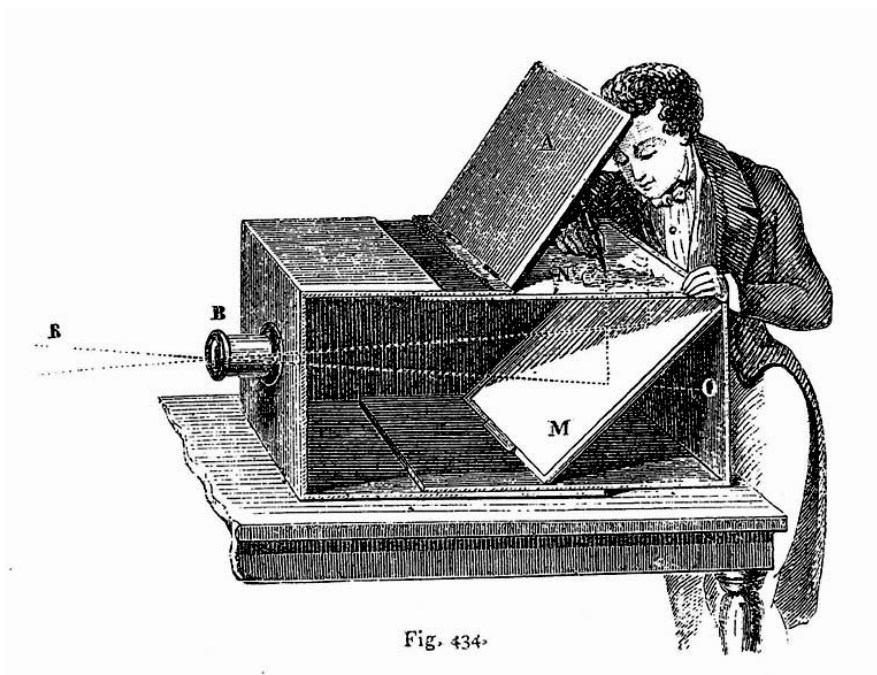


Camera Obscura, San Francisco. Wikipedia



Petapixel

Camera Lucida



Lens Based Camera Lucida, 1568

Vermeer



Vermeer, The Music Lesson, 1665

First Photograph

Oldest surviving photograph

- Took 8 hours on pewter plate (with some reports claiming it was several days)



Joseph Niepce, 1826



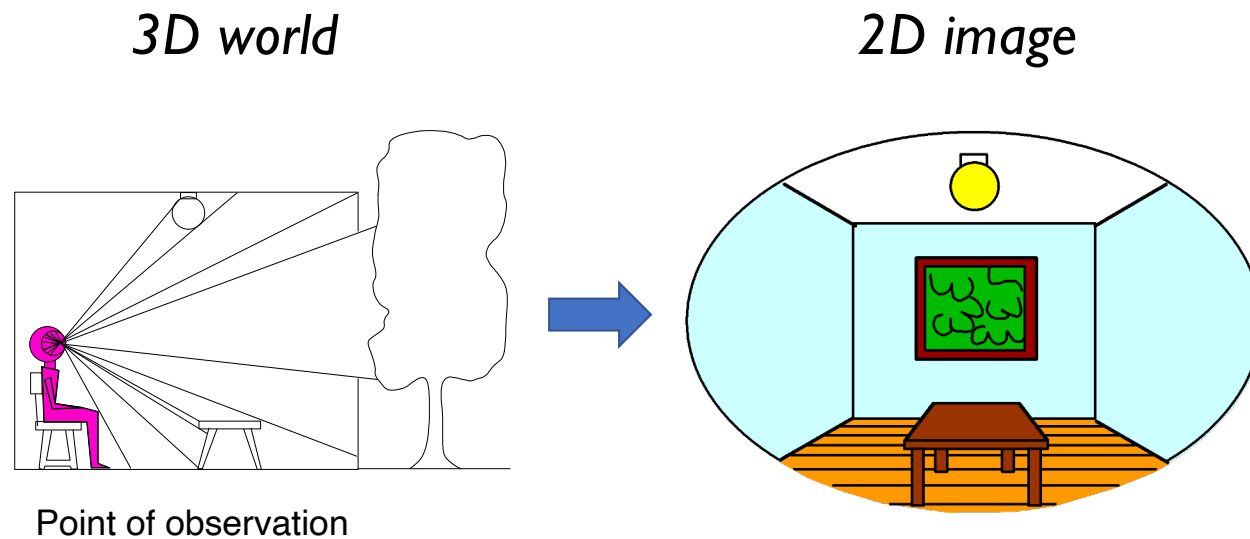
Stored at UT Austin

First Photograph with People



Boulevard du Temple, Louis Daguerre, 1838. Wikipedia.

Dimensionality Reduction Machine (3D to 2D)







Holbein's The Ambassadors - 1533

Memento Mori



Parametric (Global) Transformations

Definition

A transformation $\mathbf{T}$ is a coordinate-changing operation $p' = \mathbf{T}p$



$$p = (x, y)$$



$$p' = (x', y')$$

- What does it mean that $\mathbf{T}$ is global?
 - $\mathbf{T}$ is the same for any point p
 - $\mathbf{T}$ can be described by just a few numbers (parameters)
- For linear transformations, we can represent $\mathbf{T}$ as a matrix

$$p' = \mathbf{T}p$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \mathbf{T} \begin{bmatrix} x \\ y \end{bmatrix}$$

Common Transformations



Original

Transformed



Translation



Rotation



Scaling



Affine



Perspective

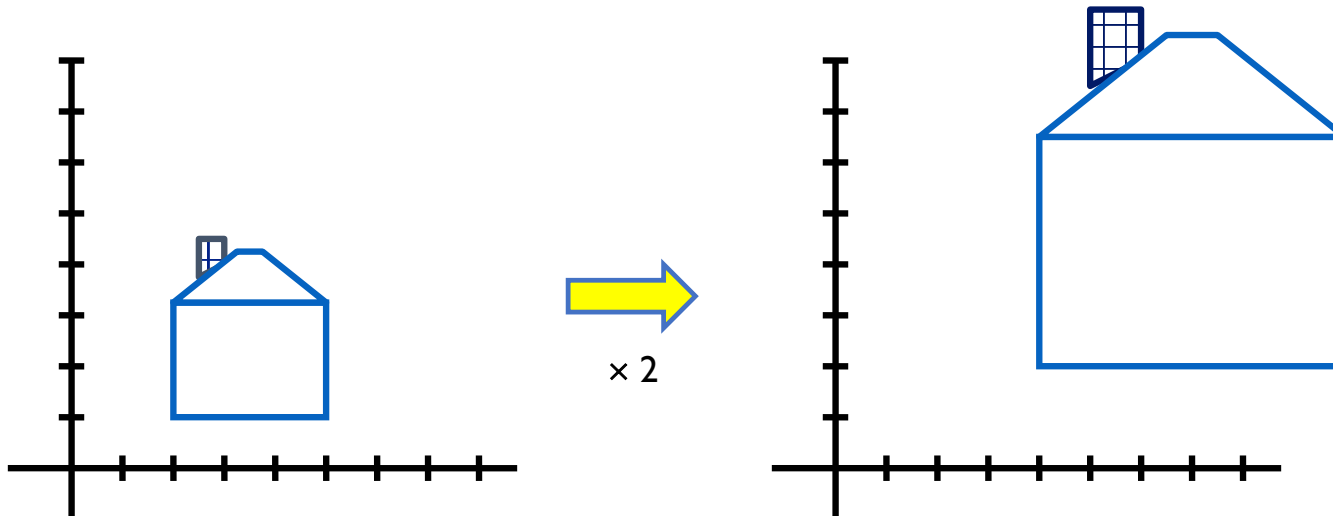
Slide credit (next few slides): A. Efros and/or S. Seitz

Scaling

Definition

Scaling a coordinate means multiplying each of its components by a scalar.

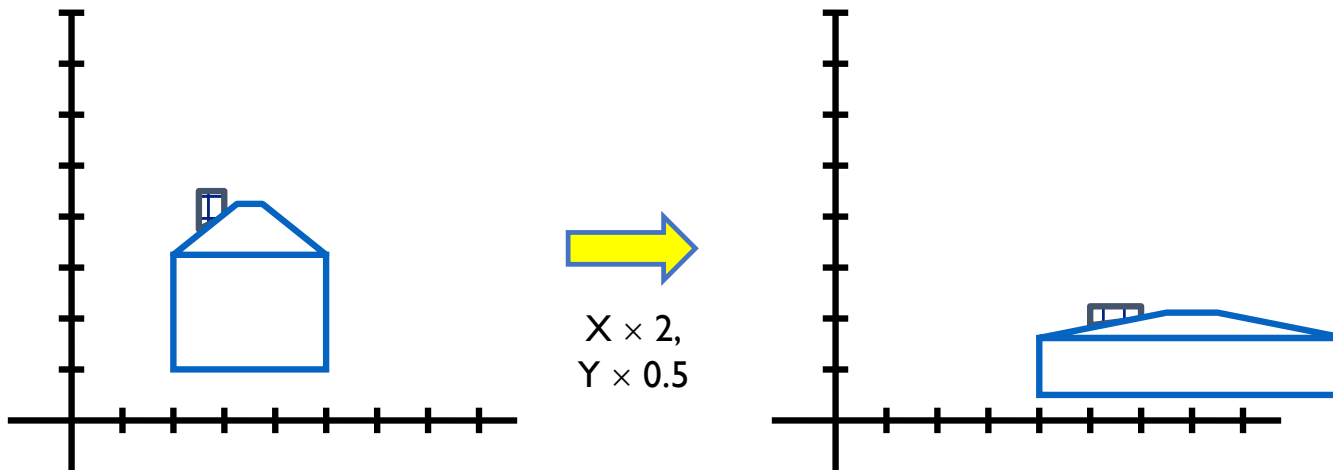
Uniform scaling means this scalar is the same for all components.



Scaling

Definition

Non-uniform scaling: different scalars per component.



Scaling

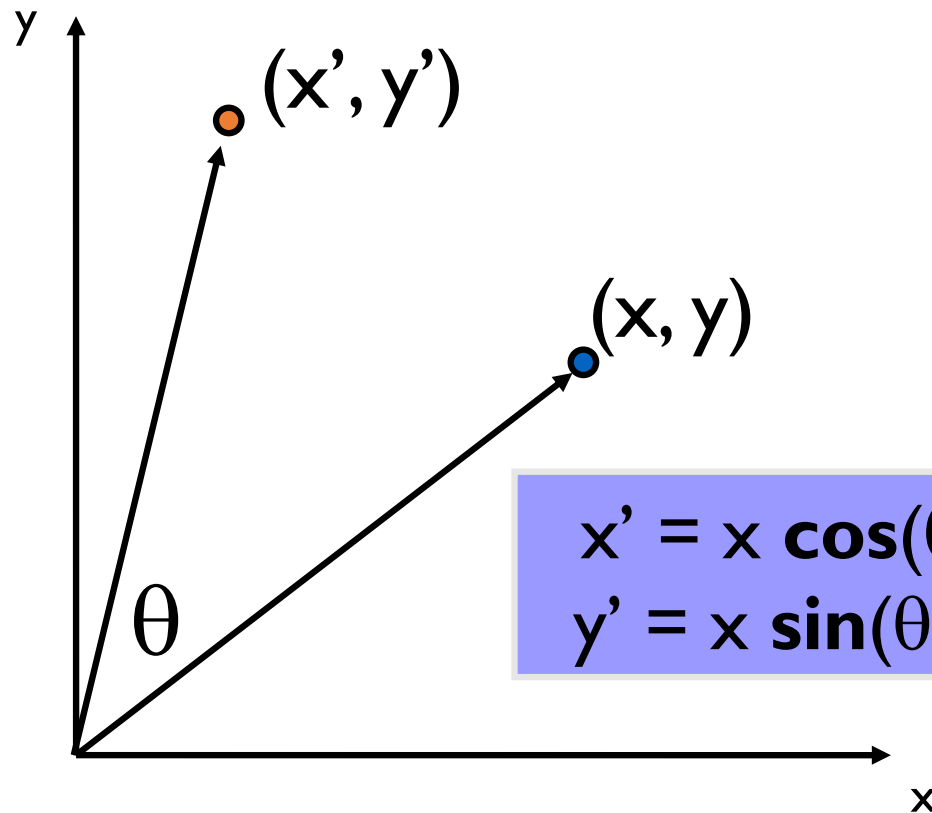
- Scaling operation:

$$\begin{aligned}x' &= ax \\ y' &= by\end{aligned}$$

- Or, in matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \underbrace{\begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix}}_{\text{scaling matrix } S} \begin{bmatrix} x \\ y \end{bmatrix}$$

2D Rotation



$$\begin{aligned}x' &= x \cos(\theta) - y \sin(\theta) \\y' &= x \sin(\theta) + y \cos(\theta)\end{aligned}$$

2D Rotation

This is easy to capture in matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \underbrace{\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}}_{\mathbf{R}} \begin{bmatrix} x \\ y \end{bmatrix}$$

Even though $\sin(\theta)$ and $\cos(\theta)$ are nonlinear functions of θ ,

x' is a linear combination of x and y

y' is a linear combination of x and y

What is the inverse transformation?

Rotation by $-\theta$

For rotation matrices $\mathbf{R}^{-1} = \mathbf{R}^T$

Basic 2D Transformations

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Scale

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & \alpha_x \\ \alpha_y & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Shear

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Rotation

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Translation

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Affine

Affine is any combination of translation, scale, rotation, and shear

Affine Transformations

Affine transformations are combinations of

- Linear transformations, and
- Translations

Properties of affine transformations:

- Origin (0,0) may move
- Lines map to lines
- Parallel lines remain parallel
- Ratios are preserved
- Closed under composition

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

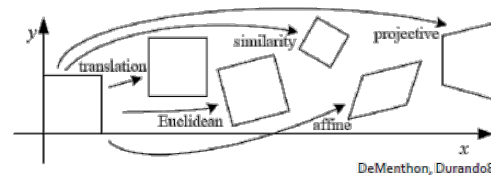
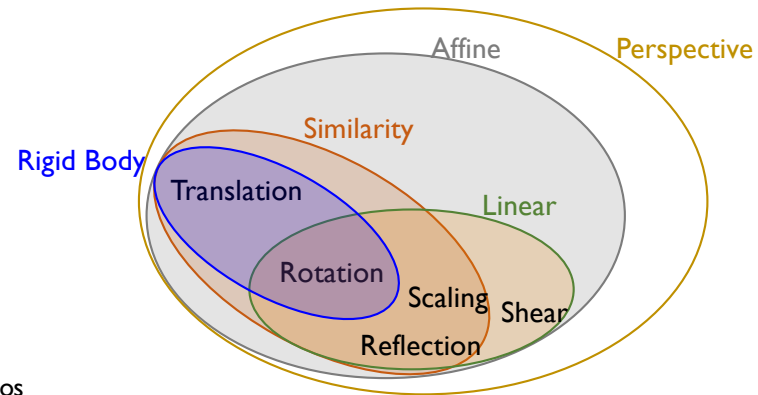
or

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

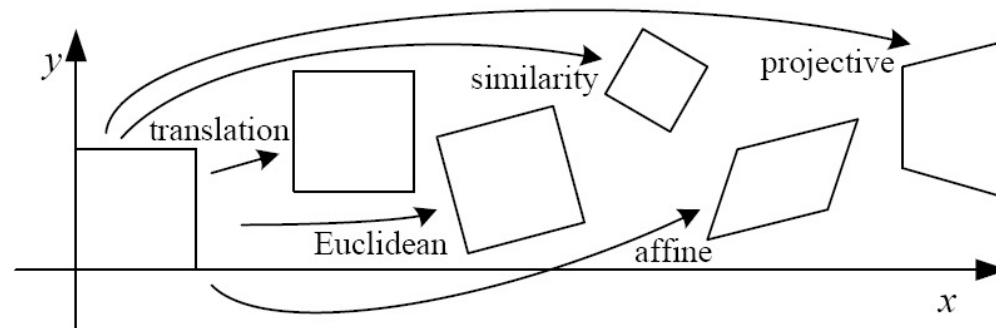
2D Transformations: Properties Preserved

Transformations can be classified based on the properties they preserve:

- **Rigid Body**
 - angles, lengths, areas
- **Similarity**
 - angles, length ratios
- **Linear**
 - linear combination
- **Affine**
 - parallel lines, length ratios, area ratios
- **Perspective**
 - collinearity, cross-ratio
- ...



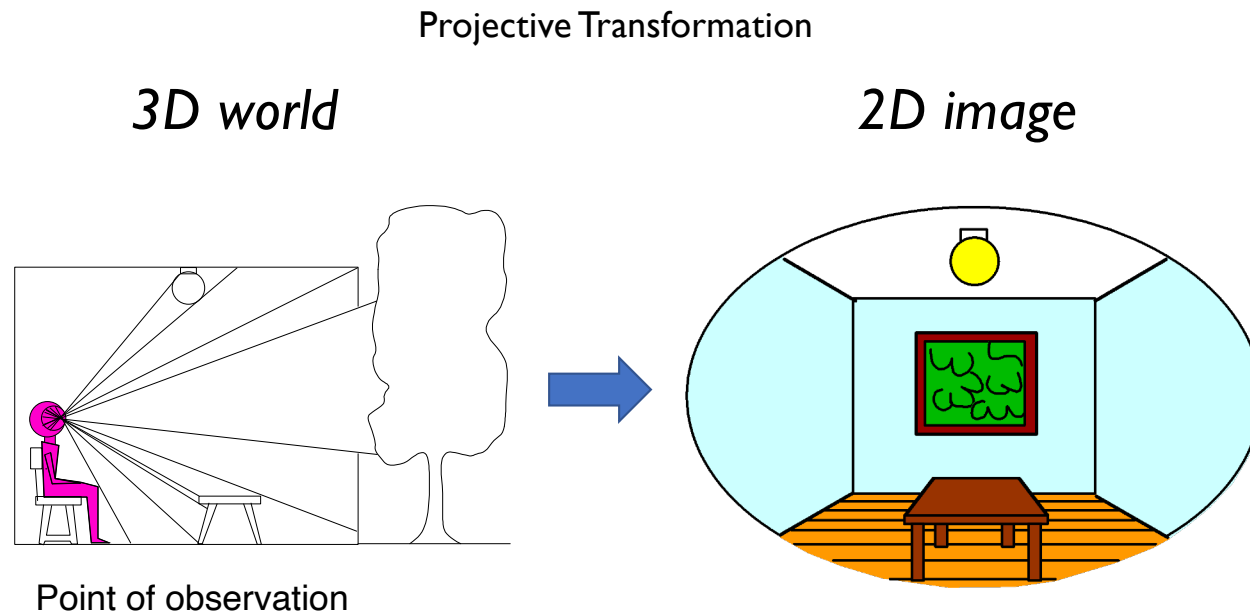
2D Transformations



Name	Matrix	# D.O.F.	Preserves:	Icon
translation	$\begin{bmatrix} I & t \end{bmatrix}_{2 \times 3}$	2	orientation + ...	
rigid (Euclidean)	$\begin{bmatrix} R & t \end{bmatrix}_{2 \times 3}$	3	lengths + ...	
similarity	$\begin{bmatrix} sR & t \end{bmatrix}_{2 \times 3}$	4	angles + ...	
affine	$\begin{bmatrix} A \end{bmatrix}_{2 \times 3}$	6	parallelism + ...	
projective	$\begin{bmatrix} \tilde{H} \end{bmatrix}_{3 \times 3}$	8	straight lines	

‘Homography’

Cameras are Projective Transformation Machines



Projective Transformations

Projective transformations are combos of

- Affine transformations, and
- Projective warps

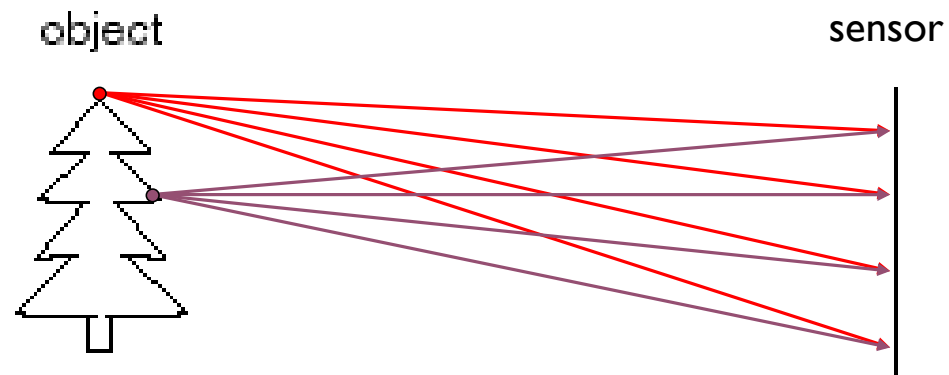
$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Properties of projective transformations:

- Origin (0,0) may move
- Lines map to lines
- Parallel lines do not necessarily remain parallel
- Ratios are not preserved
- Closed under composition
- Projective matrix is defined up to a scale (8 degrees of freedom)

Design a Camera

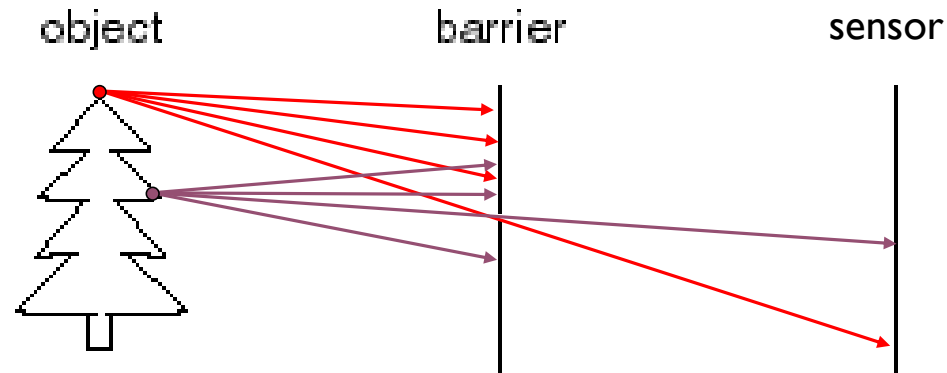
Idea 1: Put a sensor in front of an object
Do we get a reasonable image?



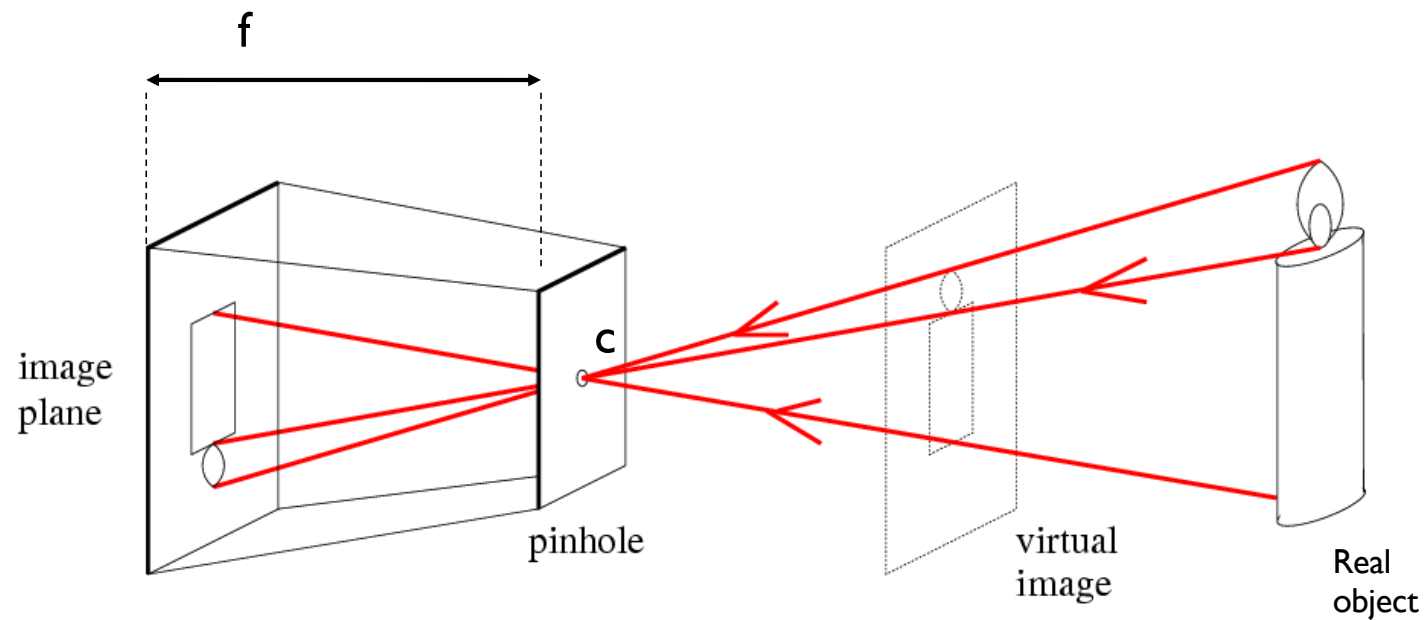
Design a Camera

Idea 2: Add a barrier to block most rays

- Pinhole in barrier
- Only sense light from one direction.
 - Reduces blurring.
- In most cameras, this **aperture** can vary in size.



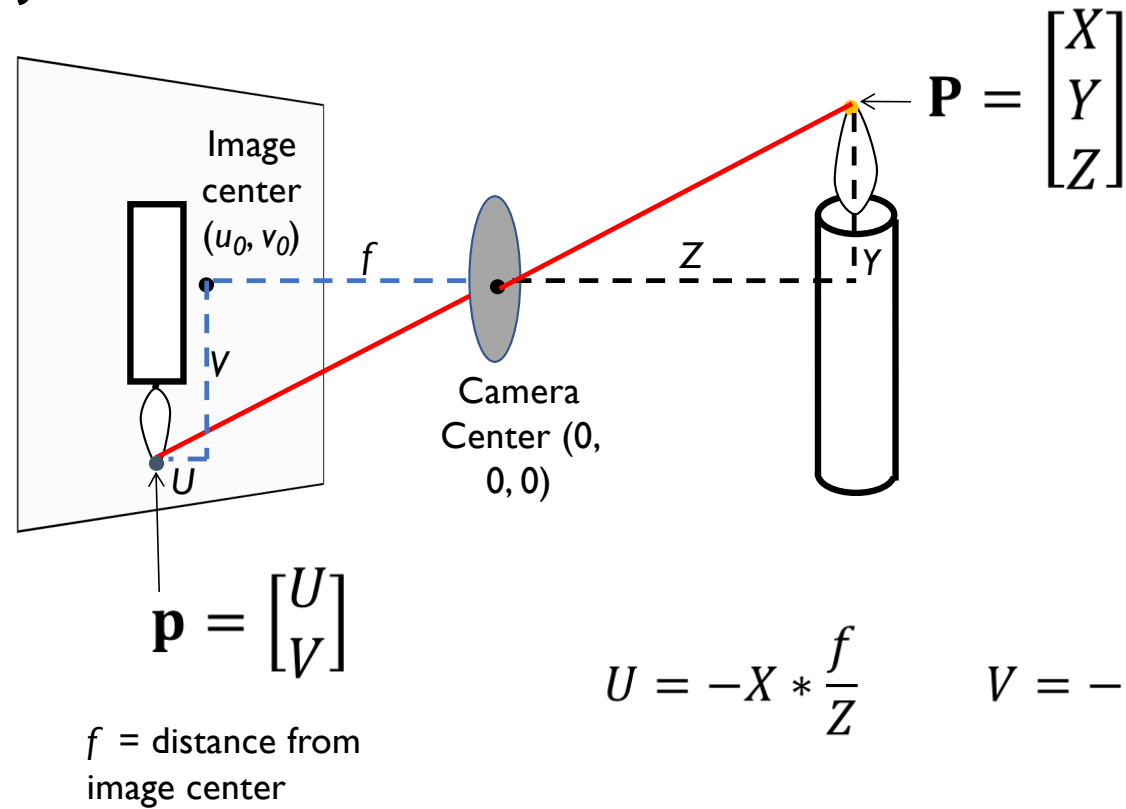
Pinhole Camera Model



f = Focal length

c = Optical center of the camera

Camera Projection

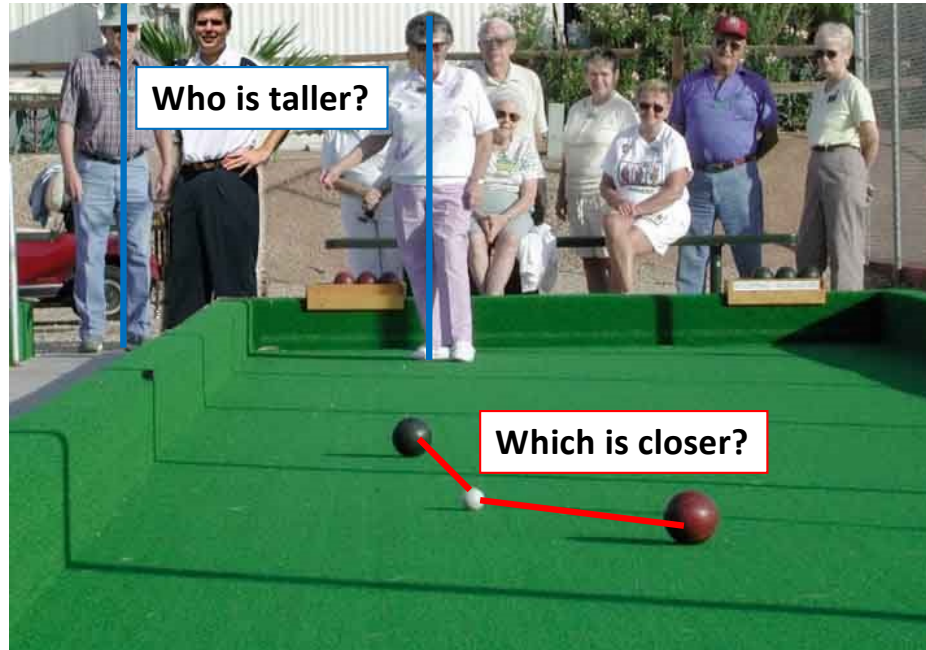


$$U = -X * \frac{f}{Z} \quad V = -Y * \frac{f}{Z}$$

What is the effect if f and Z are equal?

Projective Geometry

- Length (and so area) is lost.



Projective Geometry

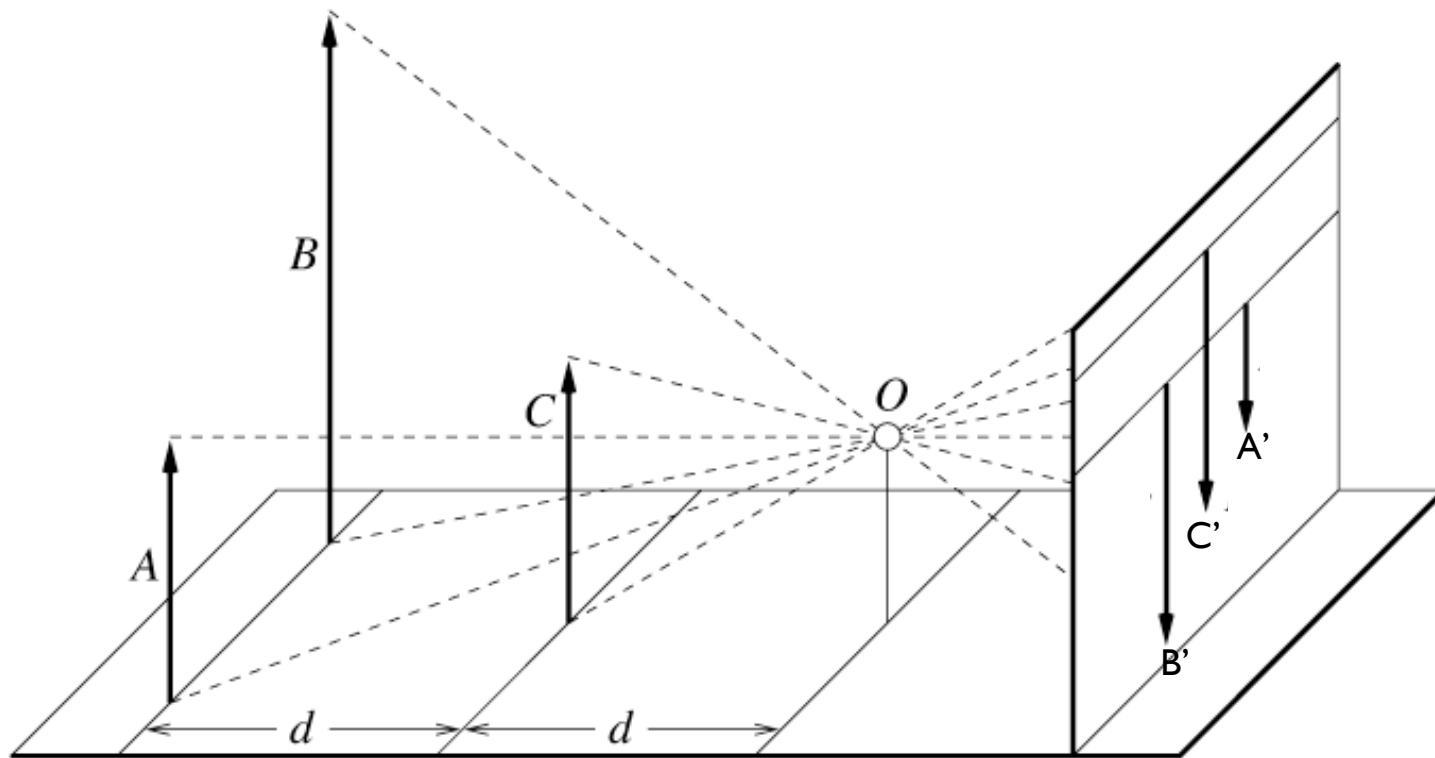
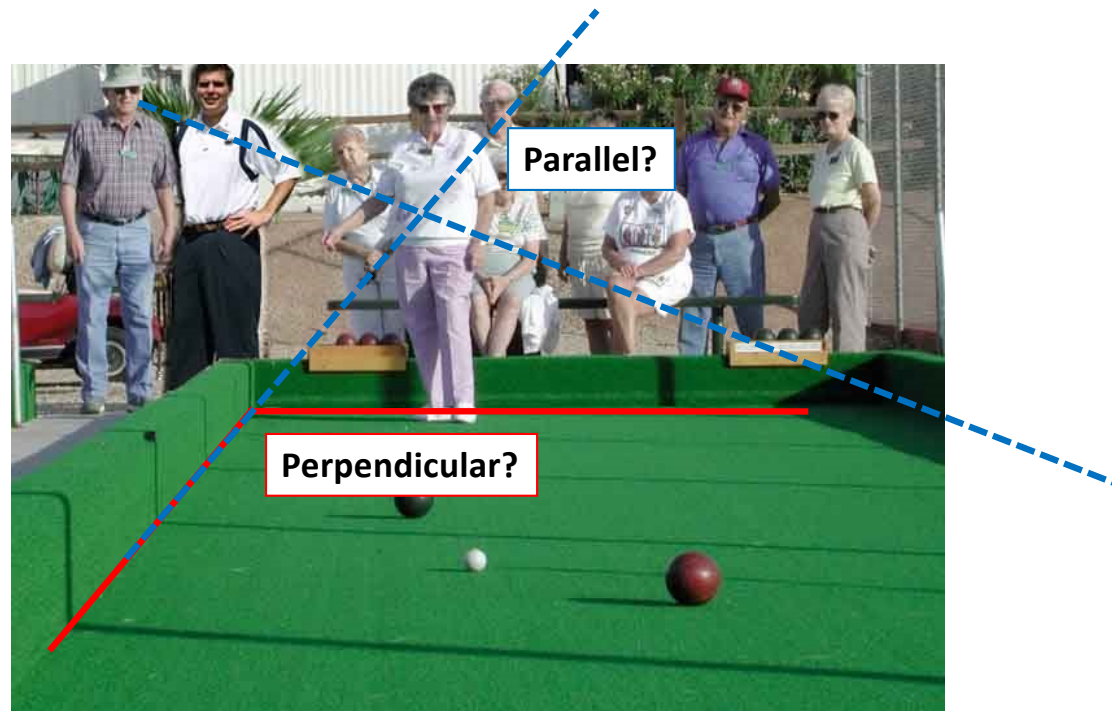


Figure by David Forsyth

Projective Geometry

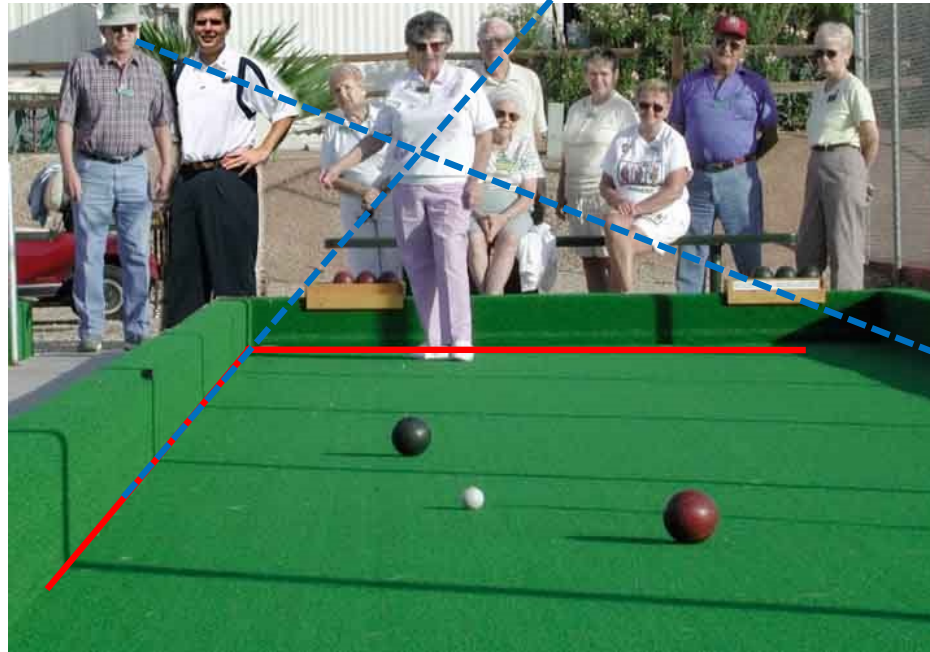
Angles are lost.



Projective Geometry

What is preserved?

- Straight lines are still straight.

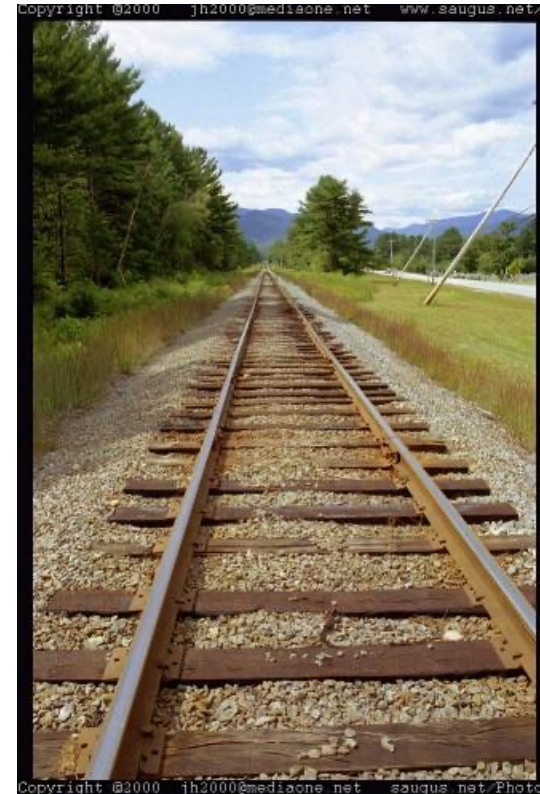
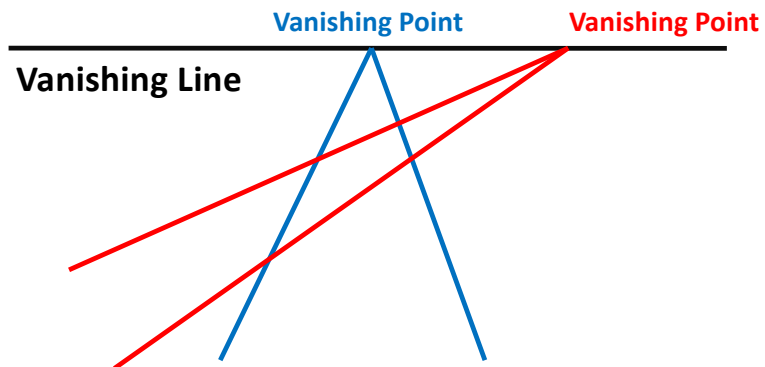


Vanishing Points and Lines

Parallel lines in the world intersect in the projected image at a “vanishing point”.

Parallel lines on the same plane in the world converge to vanishing points on a “vanishing line”.

e.g., the horizon.



Homogeneous Coordinates

Introduced by August Ferdinand Möbius

An alternative coordinate system for projective geometry

- Compare to Cartesian coordinates

Simplify formulas

- Make translation a linear transformation
- Facilitate matrix-vector formulas



Homogeneous Coordinates

Converting *to* homogeneous coordinates

$$(x, y) \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2D (image) coordinates

$$(x, y, z) \Rightarrow \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

3D (scene) coordinates

Converting *from* homogeneous coordinates

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \Rightarrow (x/w, y/w)$$

2D (image) coordinates

$$\begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} \Rightarrow (x/w, y/w, z/w)$$

3D (scene) coordinates

Homogeneous Coordinates

Scale invariance in projection space

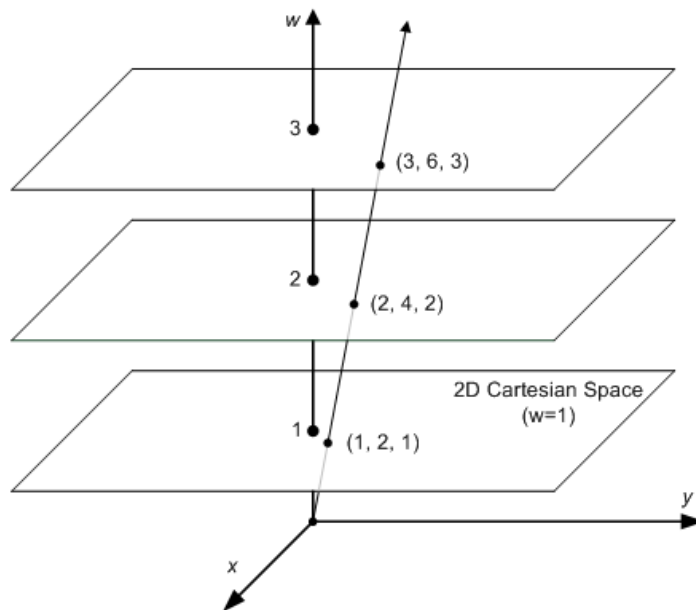
$$k \begin{bmatrix} x \\ y \\ w \end{bmatrix} = \begin{bmatrix} kx \\ ky \\ kw \end{bmatrix} \Rightarrow \begin{bmatrix} \frac{kx}{kw} \\ \frac{ky}{kw} \\ \frac{kw}{kw} \end{bmatrix} = \begin{bmatrix} \frac{x}{w} \\ \frac{y}{w} \\ 1 \end{bmatrix}$$

Homogeneous Cartesian
Coordinates Coordinates

e.g., we can uniformly scale the projective space, and it will still produce the same image -> *scale ambiguity*

Homogeneous Coordinates

- Projective
- Point becomes a line



To homogeneous

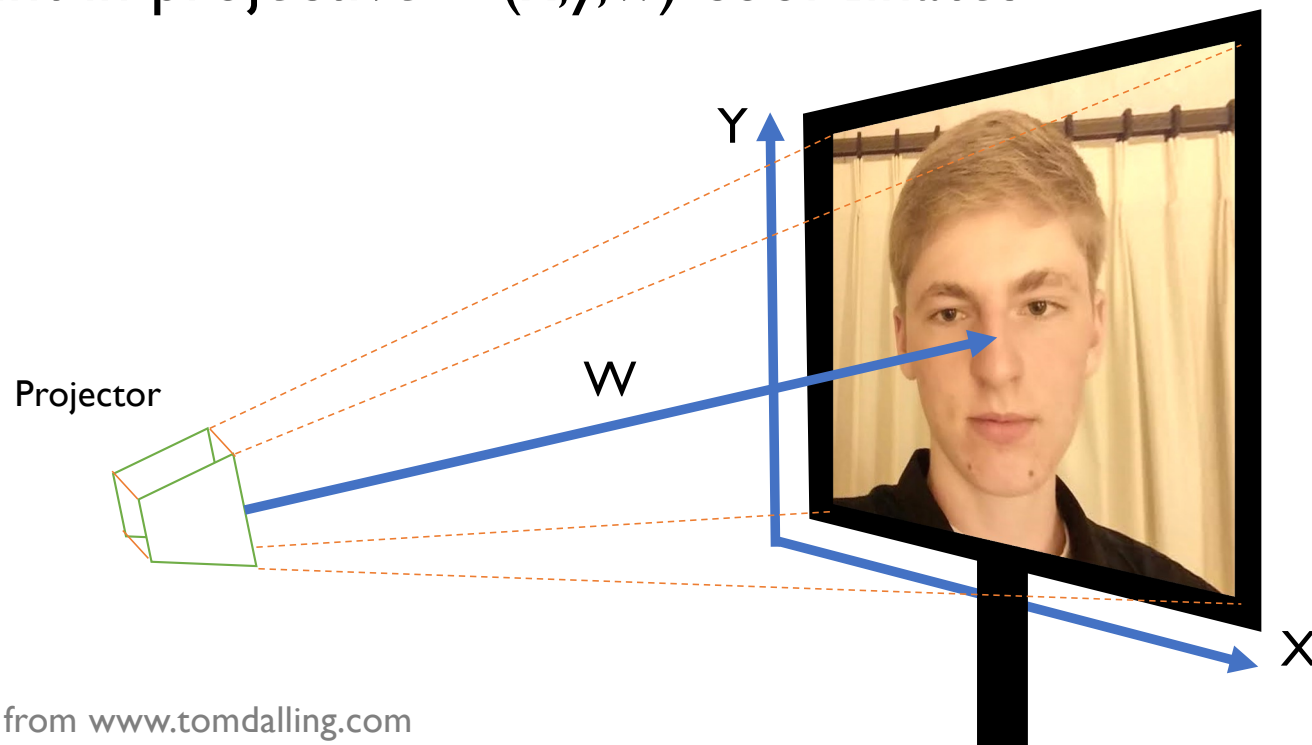
$$(x, y) \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

From homogeneous

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \Rightarrow (x/w, y/w)$$

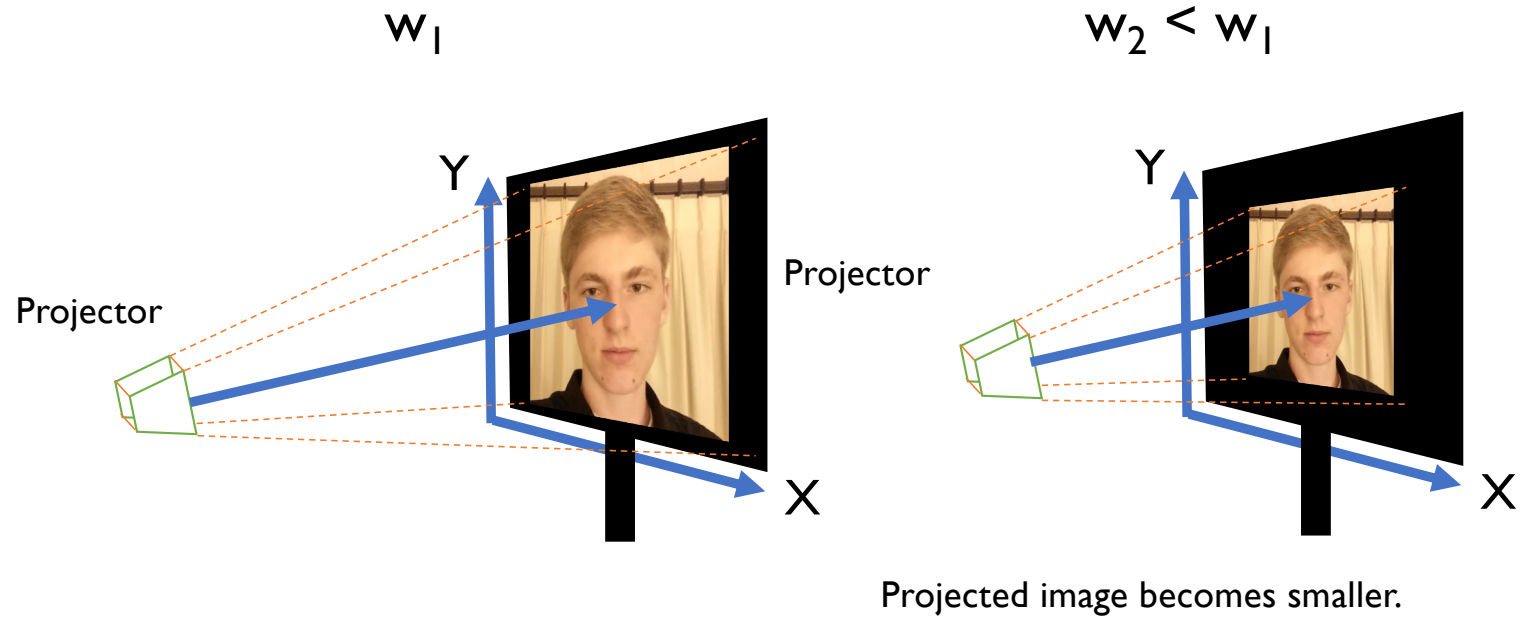
Projective geometry

- 2D point in cartesian = (x,y) coordinates
- 2D point in projective = (x,y,w) coordinates



Idea from www.tomdalling.com

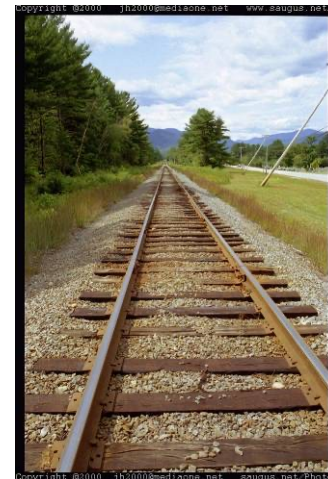
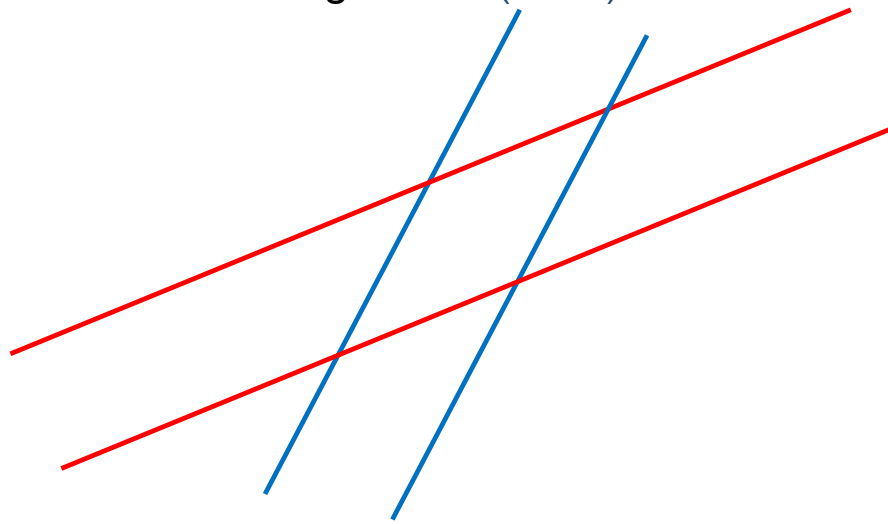
Varying w



Homogeneous Coordinates

Cartesian: (Inf, Inf)
Homogeneous: $(1, 1, 0)$

Cartesian: (Inf, Inf)
Homogeneous: $(1, 2, 0)$



Basic 2D Transformations

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Scale

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & \alpha_x \\ \alpha_y & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Shear

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Rotation

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

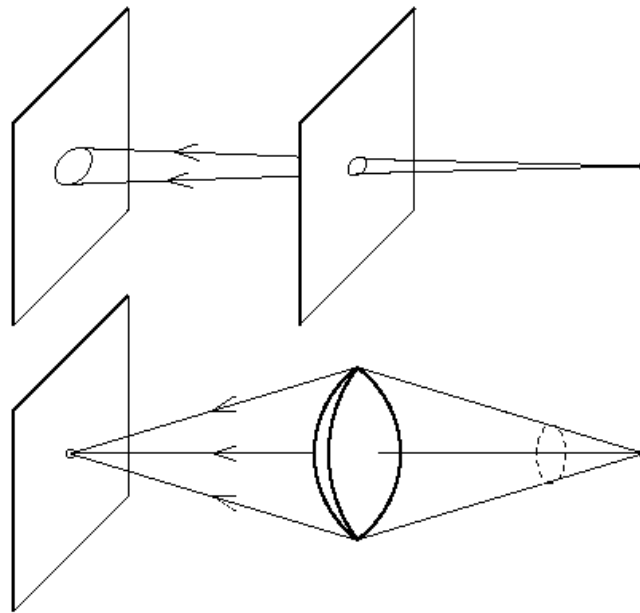
Translation

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Affine

Affine is any combination of translation, scale, rotation, and shear

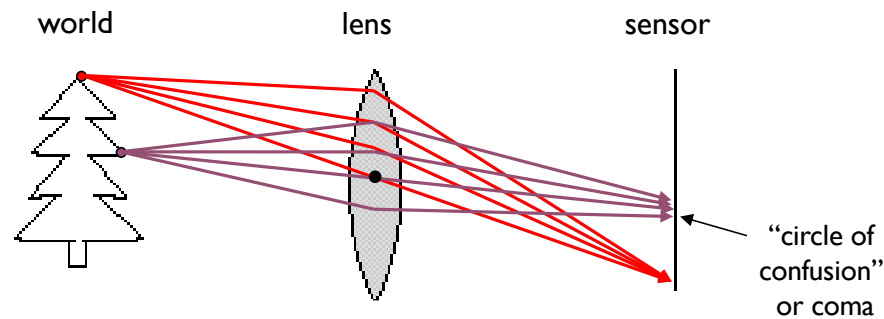
Lenses: The Need

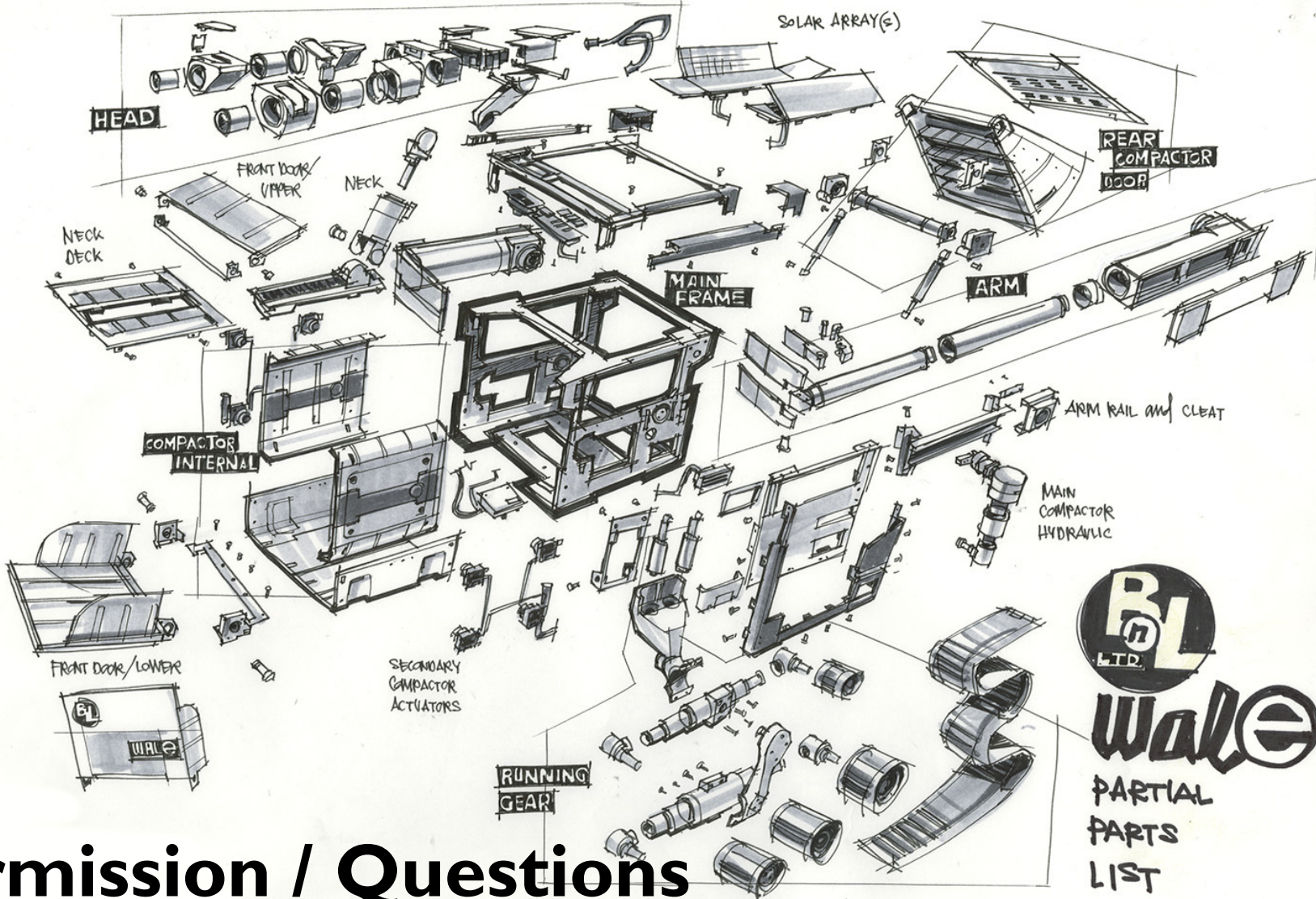


Steve Seitz

Focus and Defocus

- A lens focuses light onto the film
 - There is a specific distance at which objects are “in focus”
 - other points project to a “circle of confusion” in the image
 - Changing the shape of the lens changes this distance



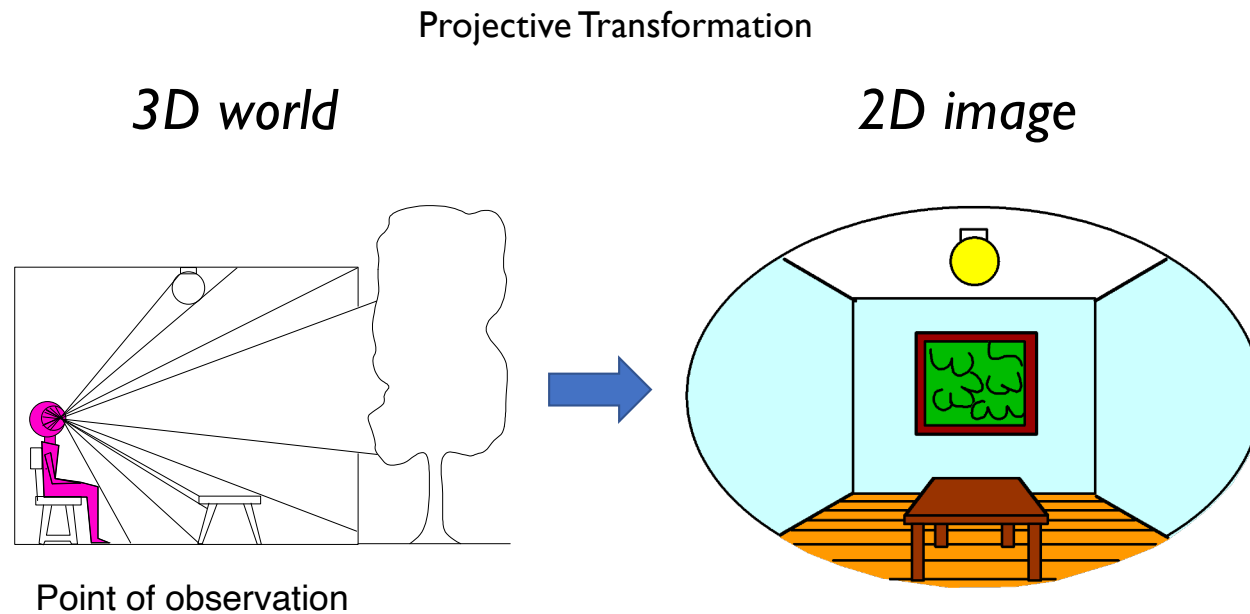


BL
LTD.
Wale
PARTIAL
PARTS
LIST

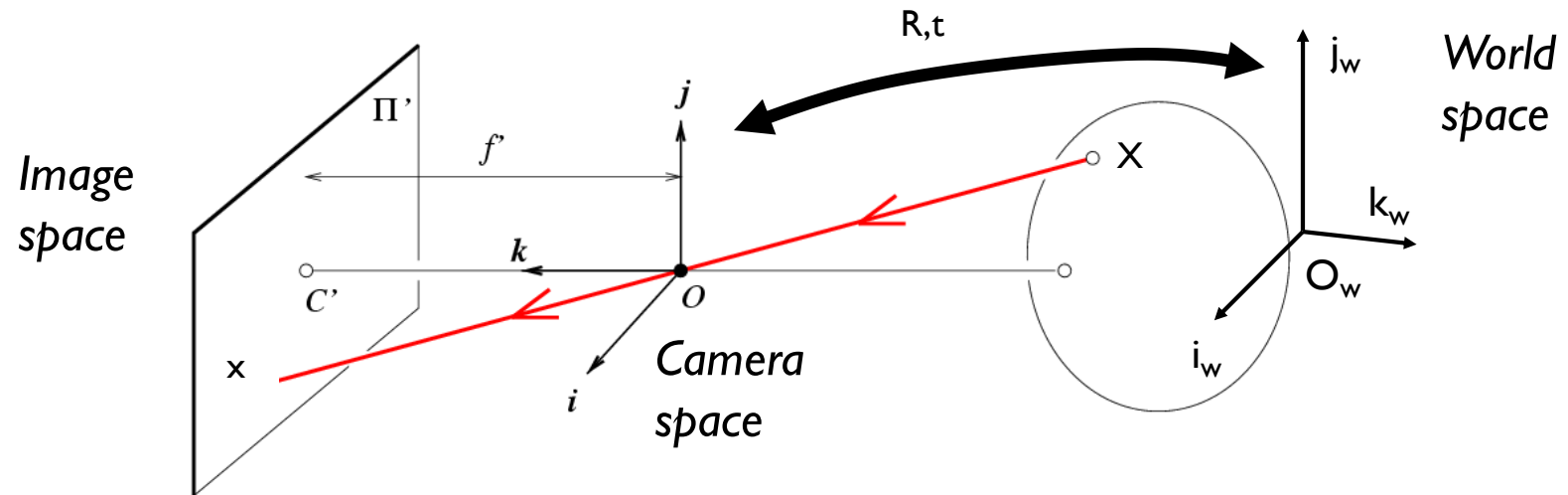
Intermission / Questions

Image Credit: Pixar Animation Studios

Cameras are Projective Transformation Machines



Camera (Projection) Matrix



$$\mathbf{x} = \mathbf{K} \underbrace{[\mathbf{R} \quad \mathbf{t}]}_{\text{Extrinsic Matrix}} \mathbf{X}$$

Extrinsic Matrix

$\mathbf{x}$: Image Coordinates: (u, v, l)

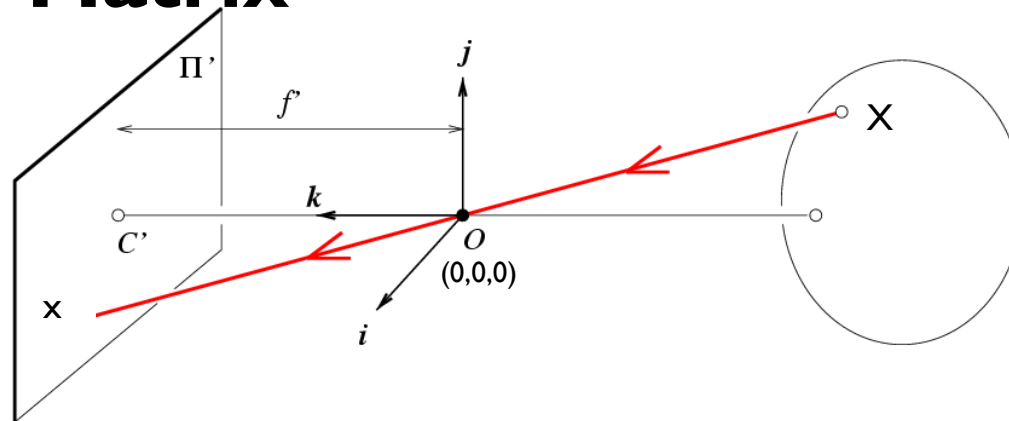
$\mathbf{K}$: Intrinsic Matrix (3×3)

$\mathbf{R}$: Rotation (3×3)

$\mathbf{t}$: Translation (3×1)

$\mathbf{X}$: World Coordinates: (X, Y, Z, l)

Projection Matrix



Intrinsic Assumptions

- Unit aspect ratio
- Optical center at (0,0)
- No skew

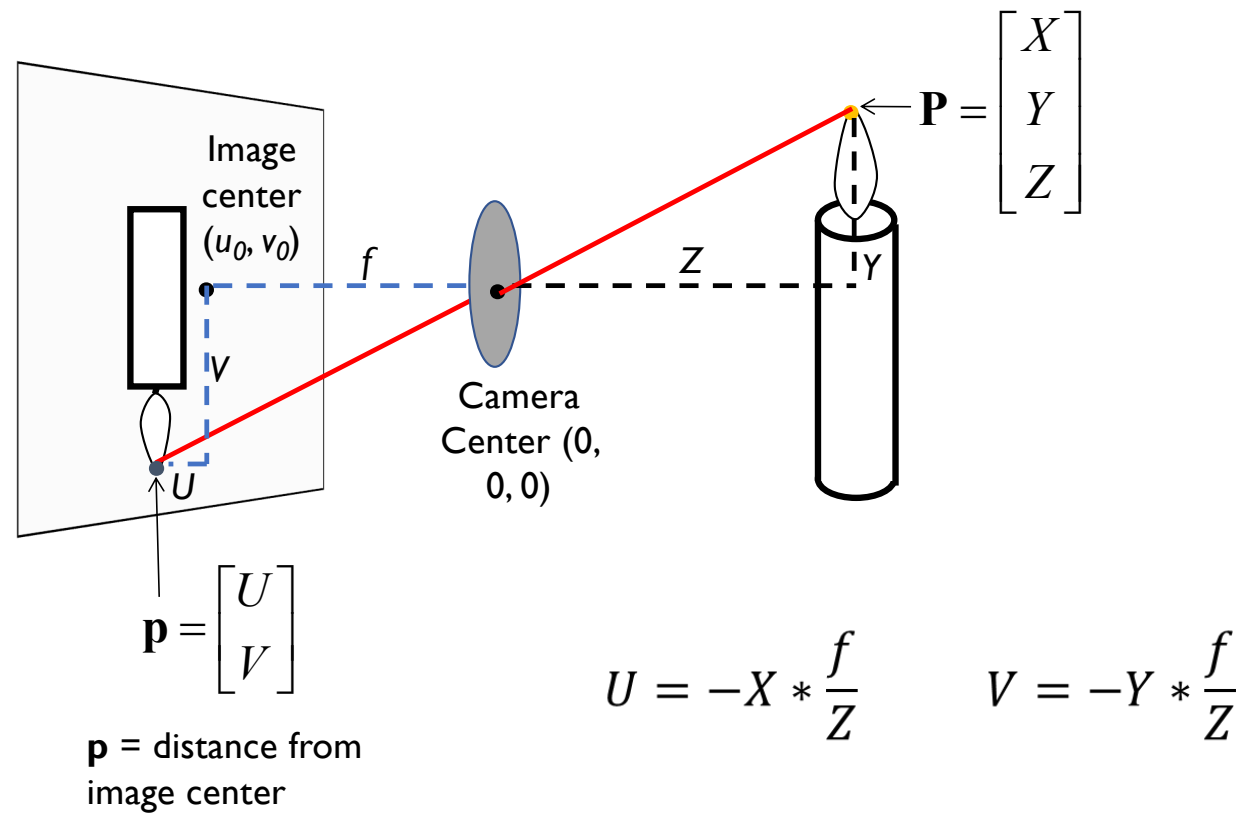
Extrinsic Assumptions

- No rotation
- Camera at (0,0,0)

$$\mathbf{x} = \mathbf{K}[\mathbf{I} \quad \mathbf{0}] \mathbf{X} \quad \Rightarrow \quad w \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

The matrix $\mathbf{K}$ is highlighted with a red dashed box in the original image.

Projection



Remove assumption: known optical center

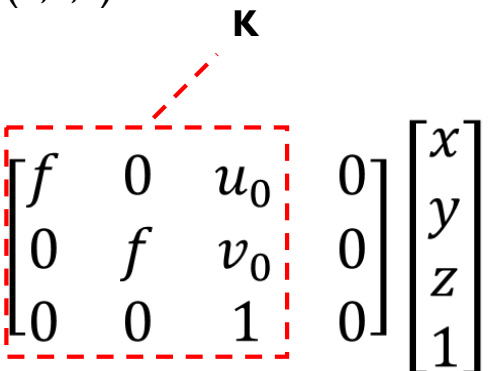
Intrinsic Assumptions

- Unit aspect ratio
- No skew

Extrinsic Assumptions

- No rotation
- Camera at (0,0,0)

$$\mathbf{x} = \mathbf{K} \begin{bmatrix} \mathbf{I} & \mathbf{0} \end{bmatrix} \mathbf{X} \rightarrow w \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f & 0 & u_0 \\ 0 & f & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



Remove assumption: equal aspect ratio

Intrinsic Assumptions

- No skew

Extrinsic Assumptions

- No rotation
- Camera at (0,0,0)

$$\mathbf{x} = \mathbf{K} \begin{bmatrix} \mathbf{I} & \mathbf{0} \end{bmatrix} \mathbf{X} \rightarrow w \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & u_0 & 0 \\ 0 & f_y & v_0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Remove assumption: non-skewed pixels

Intrinsic Assumptions

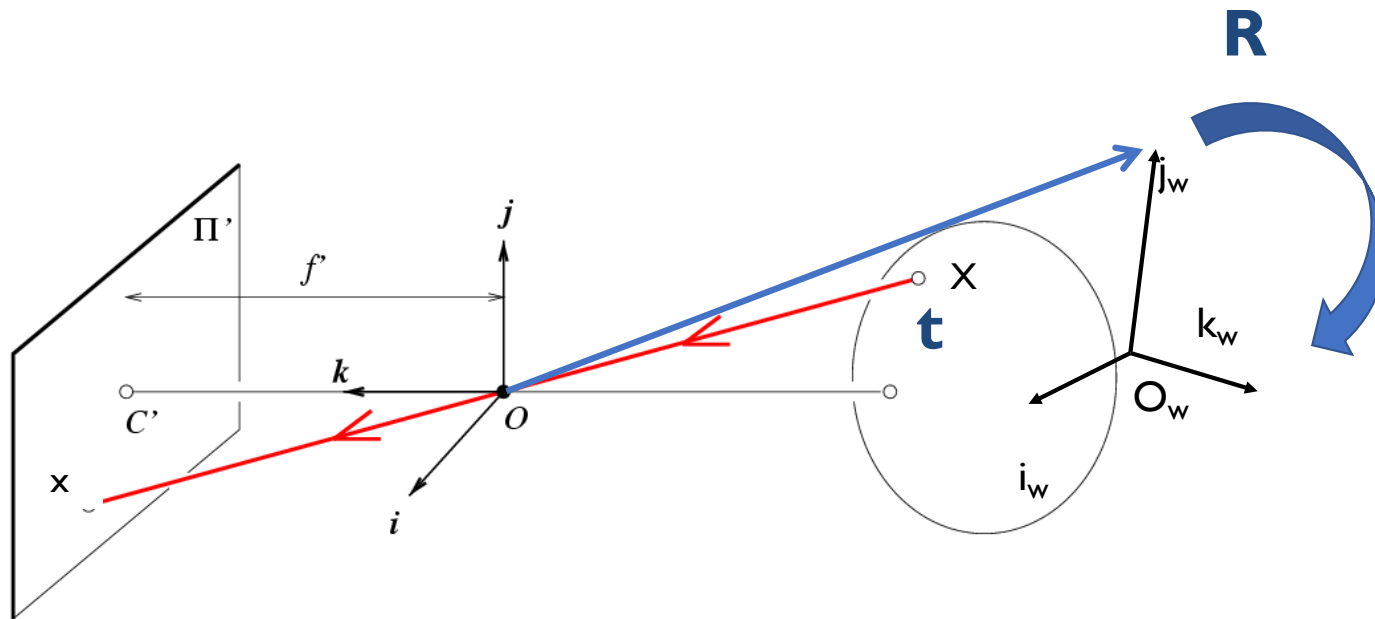
Extrinsic Assumptions

- No rotation
- Camera at (0,0,0)

$$\mathbf{x} = \mathbf{K} \begin{bmatrix} \mathbf{I} & \mathbf{0} \end{bmatrix} \mathbf{X} \rightarrow w \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & s & u_0 & 0 \\ 0 & f_y & v_0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Note: different books use different notation for parameters

Oriented and Translated Camera



Allow camera translation

Intrinsic Assumptions

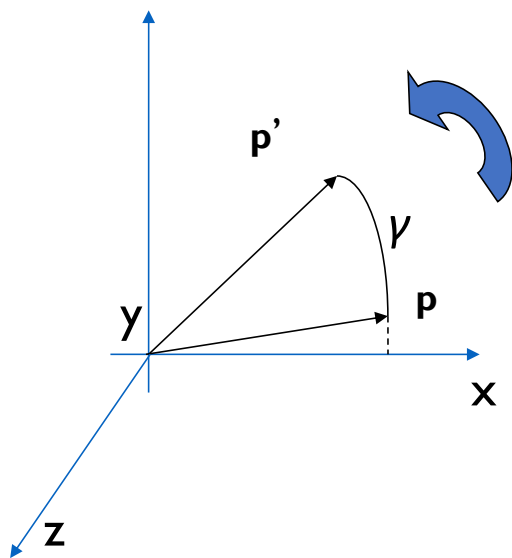
Extrinsic Assumptions

- No rotation

$$\mathbf{x} = \mathbf{K}[\mathbf{I} \quad \mathbf{t}] \mathbf{X} \quad \rightarrow \quad w \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & s & u_0 \\ 0 & f_y & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

3D Rotation of Points

Rotation around the coordinate axes, **counter-clockwise**:



$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix}$$

$$R_y(\beta) = \begin{bmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{bmatrix}$$

$$R_z(\gamma) = \begin{bmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

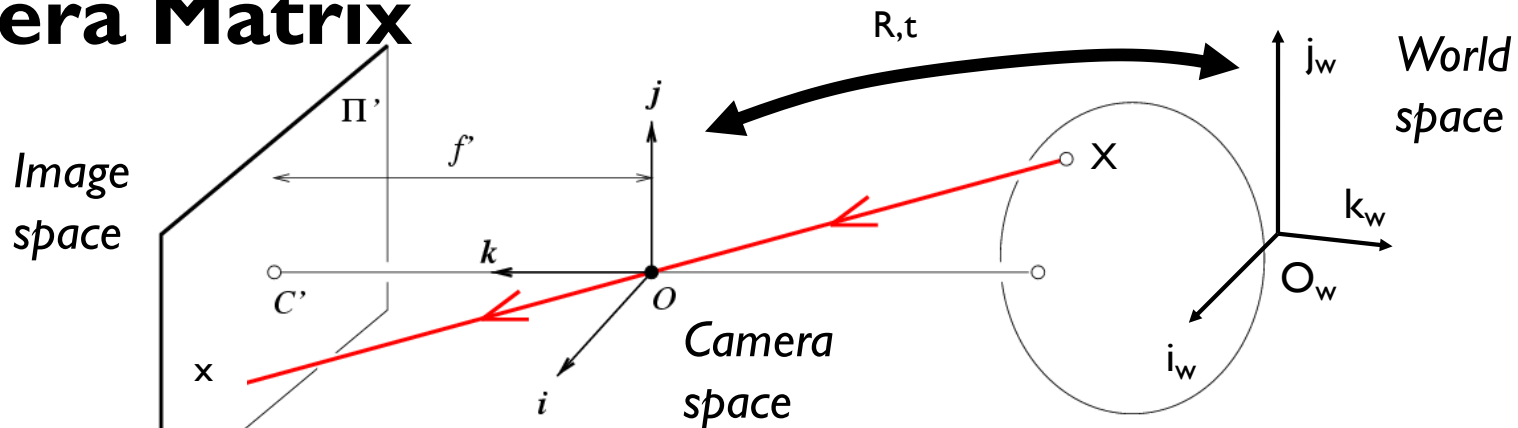
Allow camera rotation

$$\mathbf{x} = \mathbf{K}[\mathbf{R} \quad \mathbf{t}] \mathbf{X}$$



$$w \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & s & u_0 \\ 0 & f_y & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Camera Matrix



$$\mathbf{x} = \mathbf{K}[\mathbf{R} \quad \mathbf{t}] \mathbf{X}$$

Extrinsic Matrix

$\mathbf{x}$: Image Coordinates: (u, v, l)

$\mathbf{K}$: Intrinsic Matrix (3×3)

$\mathbf{R}$: Rotation (3×3)

$\mathbf{t}$: Translation (3×1)

$\mathbf{X}$: World Coordinates: (X, Y, Z, l)

$$w \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & s & u_0 \\ 0 & f_y & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Degrees of Freedom (DoF)

$$\mathbf{x} = \mathbf{K}[\mathbf{R} \quad \mathbf{t}] \mathbf{X}$$



$${}^w \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \overset{5}{\begin{bmatrix} f_x & s & u_0 \\ 0 & f_y & v_0 \\ 0 & 0 & 1 \end{bmatrix}} \overset{6}{\begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix}} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Demo – Kyle Simek

“Dissecting the Camera Matrix”

Three-part blog series

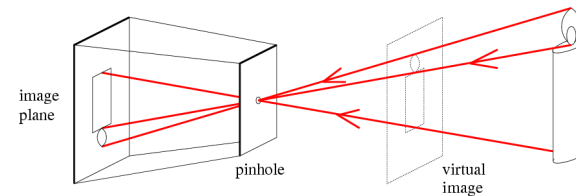
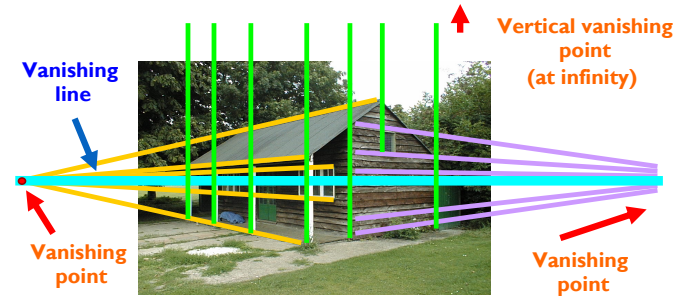
- <http://ksimek.github.io/2012/08/14/decompose/>
- <http://ksimek.github.io/2012/08/22/extrinsic/>
- <http://ksimek.github.io/2013/08/13/intrinsic/>

“Perspective toy”

- http://ksimek.github.io/perspective_camera_toy.html

Recap

- Projection loses length, area, angle, but straight lines remain straight.
- Pinhole camera model and camera projection matrix.
- Homogeneous coordinates.



$$\mathbf{x} = \mathbf{K}[\mathbf{R} \quad \mathbf{t}] \mathbf{X}$$

$$(x, y) \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

CS 322

Next episode...

- Calibration
- Stereo

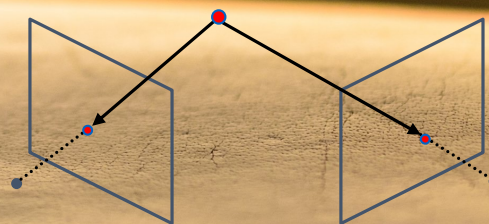
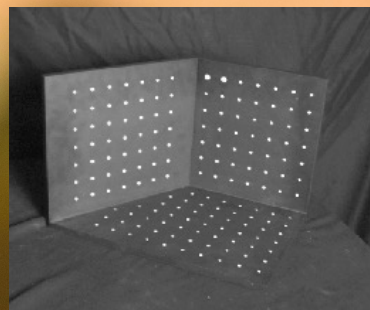


Image Credit: Wallpapercrafter.com